Project Report: Iwasawa Decomposition for $GL(n, \mathbb{R})$ in Lean 4

LeanCourse25 Project

February 17, 2026

Abstract

This project develops a matrix-level Lean 4 framework for the Iwasawa decomposition of $GL(n, \mathbb{R})$, inspired by Goldfeld's presentation. The current code formalizes core triangular-matrix infrastructure, a generalized upper-half-space model, and an API for Iwasawa factorization. The main engineering focus is a clean separation between foundational lemmas, constructive solver interfaces, and high-level decomposition statements.

1 Goal and Scope

The target mathematical statement is the decomposition

$$g = zkd, \qquad g \in GL(n, \mathbb{R}),$$

with:

- z in a generalized upper-half-space model H_n,
- k orthogonal,
- d central scalar (unit in $\mathbb{R}$).

The formalization is implemented in `LeanCourse25/Projects/Iwasawa.lean`.

2 Code Organization

The file is structured into modules:

- **Core predicates**: triangular, diagonal, unipotent, diagonal/scalar constructors.
- **Triangular lemmas**: closure under addition and multiplication.
- **Proof tools**: entrywise extensionality and helper lemmas.
- **GL layer**: matrix and invertible-matrix wrappers over $\mathbb{R}$.
- **Triangular solver API**: data types and specification for constructive elimination.
- **H_n model**: upper-unipotent times normalized positive diagonal.
- **Iwasawa API**: factorization record, existence typeclass, quotient target.

3 Architecture Overview (6 Layers)

The current file follows a strict layered architecture:

$$\text{Layer 1} \to \text{Layer 2} \to \text{Layer 3} \to \text{Layer 4} \to \text{Layer 5} \to \text{Layer 6}.$$

1. **Layer 1 (L48–262): Matrix predicates and types.**
   Definitions of `IsUpperTriangular`, `IsUpperUnipotent`, `IsDiagonal`, `IsOrthogonal`, `Mat m`, `GLn m`.

2. **Layer 2 (L276–714): Triangular solver algorithm.**
   Algorithmic elimination core: base case `solve2_kill01`, Schur recursion (`SchurStep`), and dimension-indexed solver family (`RecursiveSolveRight`).

3. **Layer 3 (L748–836): Generalized upper half-plane `Hn.H m`.**
   Data model $z = (x, y)$ with upper-unipotent x, normalized positive diagonal y, and matrix realization `z.toMat = x * diag(y)`.

4. **Layer 4 (L848–901): Factorization data type.**
   `IwasawaFactorization` stores (z, k, d) with `toMat` $= z * k * d \cdot I$.

5. **Layer 5 (L891–1487): Concrete pipeline machinery.**
   Connects solver outputs to factorization data via `ConcretePipeline`, `FromTriangularSolver`, and `ConstructiveCore`.

6. **Layer 6 (L1490–2230): Theorems and $GL(n, \mathbb{Z})$-action.**
   Global existence packaging, uniqueness, quotient equivalence, arithmetic group action, and Siegel-set statement.

Layer 6 Status Table

Theorem	Line	Status
`unique_z_of_two_factorizations`	L1790	DONE
`Hn_equiv_quotient`	L2044	DONE
`GLZ_mulAction_Hn`	L2252	DONE
`siegel_fundamental_domain`	L2279	`sorry`

4 Core Formal Definitions

For $N = \text{Fin } m$, the project defines:

- `IsUpperTriangular`, `IsLowerTriangular`, `IsDiagonal`,
- `IsUpperUnipotent`, `IsLowerUnipotent`,
- `diag : (N -> alpha) -> Matrix N N alpha`,
- `scalarMat : alpha -> Matrix N N alpha`.

These abstractions provide a uniform basis for all later constructions.

5 Key Proved Lemmas

The file includes reusable structural lemmas:

- `upper_add`, `lower_add`,
- `upper_mul`, `lower_mul`,
- `diagonal_upper`, `diagonal_lower`,
- `upper_eq_lower_implies_diagonal`.

These are the main proof-theoretic building blocks for uniqueness arguments and elimination workflows.

6 Constructive Solver Layer

In namespace `TriangularSolver`, the code introduces:

- `UpperUnipotent` as a structure carrying both data and proofs,
- `SolveRightSpec` as a solver contract for equations of the form $UA = B$,
- completeness formalization `SolveRightSpec.Complete` and impossibility result `not_complete`,
- concrete 2×2 lemmas `solve2_kill01`, `solve2_upperUnipotent`, `solve2_lowering_data`,
- a field-level solver `solveRight2Field` and its real specialization `solveRight2Real`,
- recursive infrastructure `SchurStep`, `stepOfSchur`, `recursiveSolveRightOfSchur`, and `RecursiveSolveRight`.

This design isolates computational elimination from downstream decomposition logic.

7 Generalized Upper Half-Space H_n

The file models H_n as structured data:

$$z = xy,$$

where:

- x is upper-unipotent (`Hn.UpperUnipotent`),
- y is normalized positive diagonal (`Hn.PosDiagNorm`).

The map `Hn.H.toMat` gives the underlying matrix representation.

8 Iwasawa API

The final API layer defines:

- `IwasawaFactorization` with fields (z, k, d),
- `ConcretePipeline` (algorithmic data: `xOf`, `yOf`, `qOf`, `dOf`, `correct`),
- bridge layers `FromTriangularSolver` and `ConstructiveCore`,
- `HasIwasawa` (typeclass) and immediate theorem `exists_factorization`,
- uniqueness theorem `unique_z_of_two_factorizations`,
- quotient objects `OZ_subgroup`, `Hquot`, `Hn_equiv_quotient`.

This makes downstream usage independent of the eventual constructive proof of the instance.

Core Contract: `HasIwasawa`

The central abstraction is:

- a constructor
  $$\texttt{iwasawa : GLn m -> IwasawaFactorization m},$$
  which assigns factors (z, k, d) to each invertible matrix g;
- a correctness theorem
  $$\texttt{correct : } \forall g,\ (\texttt{iwasawa}\ g).\texttt{toMat} = g.$$

So `HasIwasawa` is exactly "existence + reconstruction correctness". All downstream theorems can depend on this interface without committing to a specific algorithm.

Why `exists_factorization` is immediate

The proof

```
theorem exists_factorization [HasIwasawa m] (g : GLn m) :
    exists fac, fac.toMat = g
```

is one-step logical unpacking of the class fields:

1. choose the witness
   $$\texttt{fac := HasIwasawa.iwasawa g};$$
2. remaining goal is exactly
   $$\texttt{fac.toMat = g},$$
   which is `HasIwasawa.correct g`;
3. `simpa` finishes after unfolding the chosen witness.

In short, the theorem is a canonical projection from the typeclass data.

Full Construction Pipeline (Exact Code Path)

Below is the concrete path in your code from input g to the data used by `HasIwasawa`.

1. **Input.** Start from
$$g : \texttt{GLn m}.$$
Target:
$$\texttt{fac : IwasawaFactorization m}$$

2. **Base elimination at dimension 2.** Use `solve2_kill01` to kill entry $(0,1)$, then package as `UpperUnipotent` via `solve2_upperUnipotent`, and package $B = U*A$ via `solve2_lowering_data`. This yields the concrete base `solveRight2Field` and `solveRight2Real`.

3. **Recursive solver family.** Provide Schur-step data `SchurStep` and build higher-dimensional solvers with `stepOfSchur` and `recursiveSolveRightOfSchur`, giving `RecursiveSolveRight`.

4. **Algorithmic factor data at fixed dimension.** Use `ConcretePipeline m`, whose fields are exactly the data production functions: `xOf`, `yOf`, `qOf`, `qOrth`, `dOf`, plus a reconstruction proof `correct`.

5. **Package factors.** From any pipeline `P : ConcretePipeline m` and g:
   - `zOf P g : Hn.H m`,
   - `facOf P g : IwasawaFactorization m`,
   - `facOf_correct P g : (facOf P g).toMat = g`.

   This already gives `exists_factorization_of_pipeline`.

6. **Dimension recursion for pipelines.** Use `RecursiveMkPipeline` and `RecursiveConcretePipeline` to get pipelines at arbitrary dimensions $m \geq 2$, with theorem `RecursiveConcretePipeline.exists_factoriza`

7. **Bridge from solver recursion to pipeline recursion.** Use `FromTriangularSolver` (fields: `solver`, `mkPipeline`), or equivalently raw witness packaging via `RawMkPipeline`. This yields theorems `exists_raw_ge2`, `exists_factorization_ge2`, and `Fact`-versions.

8. **Single constructive package.** Bundle everything into
$$\texttt{ConstructiveCore := \{ solverBase, solverSchur, pipelineBuilder \}}.$$
   Then derive: `solverRec`, `bridge`, `exists_factorization_fact`, and `toHasIwasawaFact`.

9. **Typeclass extraction.** At this point, obtain
$$\texttt{HasIwasawa m}$$
   via `ConcretePipeline.toHasIwasawa`, `FromTriangularSolver.toHasIwasawaFact`, or `ConstructiveCore.to`
   With a registered global core (`HasConstructiveCore`), the instance `hasIwasawaFact` is automatic.

10. **Final theorem as projection.** Once [`HasIwasawa m`] is available, the theorem `exists_factorization` is immediate by taking witness `HasIwasawa.iwasawa g` and applying `HasIwasawa.correct g`.

Notation map used in this report:
$$(U, M, D) \longrightarrow (x, y, d) \longrightarrow (z, k, d),$$
where $z = (x, y)$ is exactly the `Hn.H`-object stored in `IwasawaFactorization`.

9 Current Status and Remaining Tasks

The infrastructure and API are in place and compile. Major completed milestones now include:

- constructive base solver and recursive solver framework (`TriangularSolver`),
- concrete and recursive pipeline packaging (`ConcretePipeline`, `ConstructiveCore`),
- uniqueness of the z-factor (`unique_z_of_two_factorizations`),
- quotient construction and equivalence map (`OZ_subgroup`, `Hquot`, `Hn_equiv_quotient`).

At the moment, the remaining unfinished theorem in this file is:

- `siegel_fundamental_domain` (currently marked with `sorry`).

10 Conclusion

This project establishes a strong formal skeleton for Iwasawa decomposition in Lean 4: core matrix algebra, triangular proof infrastructure, a solver interface, and a scalable API. The architecture is suitable for extending from specification-level statements to full constructive proofs.

Reference

D. Goldfeld, *Automorphic Forms and L-Functions for the Group GL(n, R)*, Cambridge Studies in Advanced Mathematics, vol. 99, 2006.